

Scrum in the AI “Dark Factory” Era

Executive summary

Scrum remains **relevant**, but its **center of gravity shifts** from managing *human* execution capacity to managing *organizational intent, risk, and learning* in a world where execution is increasingly performed by AI systems (copilots and autonomous agents). Scrum’s enduring value is its **empirical control system**—transparency, inspection, and adaptation—explicitly designed to control risk and optimize predictability in complex work. ¹

The most robust evidence since 2018 suggests that AI can substantially increase **developer task throughput** in many contexts, but these gains are **not guaranteed to translate** into improved end-to-end delivery performance unless organizations make complementary investments (platforms, small-batch discipline, data/knowledge readiness, and clear AI governance). For example, controlled and field studies show meaningful productivity uplifts from AI coding assistants (e.g., **55.8% faster completion** in one controlled experiment and **~26% more completed tasks** in combined field experiments across thousands of developers). ² At the same time, DORA’s 2024 research-based analysis indicates that a **25% increase in AI adoption** is associated with improvements in *documentation quality, code quality, and review speed*, yet can correlate with **worse delivery throughput and stability**—a warning that faster code production can increase systemic instability if quality gates, batching, observability, and governance do not keep pace. ³

“Dark factory” software development—**autonomous agents that build, test, and ship with minimal/no human code writing or review**—is emerging primarily from industry experimentation rather than peer-reviewed longitudinal studies. High-profile narratives claim multi-x productivity improvements, and some organizations describe “non-interactive” development where humans write specs/scenarios and agents converge on working code. However, these accounts remain **unevenly verifiable** and should be treated as *frontier case studies* rather than settled evidence. ⁴ Meanwhile, research benchmarks (e.g., SWE-bench) and agent systems (e.g., SWE-agent, AutoCodeRover) show that autonomous issue resolution is advancing but remains bounded by verification, context retrieval, and reliability constraints—exactly the kinds of constraints Scrum-like inspection/adaptation loops address organizationally. ⁵

The most actionable conclusion is that Scrum’s **core artifacts and commitments** (Product Goal, Sprint Goal, Definition of Done) remain valuable, but they must expand to cover **AI-specific quality and governance** (traceability, provenance, security, evaluation harnesses, and operational guardrails). ⁶ Some widely practiced Scrum “extras” (not required by the Scrum Guide)—especially **story-point-centric estimation** and **meeting-heavy status synchronization**—are the most likely to become obsolete or minimized when AI automates execution and reporting.

Recommended direction: evolve Scrum into an **Intent-to-Increment operating cadence** where (a) “backlog items” become **testable intent packages** (specs + scenarios + non-functional constraints), (b) sprint cycles compress or move toward **continuous flow** depending on risk, and (c) success metrics shift from velocity to **outcome + flow + reliability + governance** (DORA + SPACE + AI safety metrics). ⁷

Definitions and automation maturity

AI-augmented development

AI-augmented development is **human-led** software engineering where AI assists with micro-tasks such as code completion, refactoring suggestions, test generation, documentation drafts, and summarization. Evidence for productivity effects is strongest in this tier, with both controlled experiments and field experiments finding measurable improvements under certain conditions. ²

AI dark factory software development

“Dark factory” (or “dark software factory”) borrows from manufacturing’s “lights-out” concept—**fully automated production** with minimal human presence. Applied to software, it refers to environments where **autonomous agents build, test, and ship** around the clock while humans focus on defining business intent and overseeing outcomes. ⁴

Important nuance: credible “dark factory” descriptions do **not** imply “no governance.” They typically relocate effort from hand-coding to: (a) writing precise intent, (b) building validation harnesses and “holdout” scenarios, and (c) monitoring reliability and risk. ⁴

Levels of automation in software delivery

There is no single industry-standard “levels of automation” scale for software engineering comparable to driving automation standards. In practice, organizations are converging on **maturity ladders** that resemble “from autocomplete to software factory.” One influential practitioner model describes tiers from manual coding through AI pair-programming to agent-led development where humans increasingly manage and validate rather than author code. ⁸

A pragmatic synthesis (for analysis and governance) is:

```
flowchart LR
    L0["Level 0: Human-only\nManual coding + manual reviews"] --> L1["Level 1: AI assist\nAutocomplete, snippets, docs/tests drafts"]
    L1 --> L2["Level 2: AI pair-programmer\nHumans drive; AI proposes larger units"]
    L2 --> L3["Level 3: Agentic delivery (human-in-loop)\nAgents implement from intent; humans gate merges/releases"]
    L3 --> L4["Level 4: 'Dark factory' (human-on-loop)\nAgents implement/test/deploy; humans steer intent + monitor outcomes"]
    L4 --> L5["Level 5: Autonomous product ops\nAgents also detect opportunities, propose roadmaps,\nrun experiments—humans set policy + ethics + constraints"]
```

This maturity framing matters because Scrum’s “best shape” differs by level: **timeboxed sprints** are often useful at Levels 0–3 (learning cadence), while **continuous flow + high-assurance controls** becomes more attractive at Levels 3–5 when execution is near-continuous and the bottleneck becomes intent/verification. ⁹

Current evidence since 2018

Productivity and throughput

High-quality empirical evidence supports **local productivity gains** from AI coding assistants, but also reveals **variance by context, experience, and organizational conditions**.

A controlled experiment on GitHub Copilot found the treatment group completed a programming task **55.8% faster** than control. ¹⁰ A larger set of real-world randomized controlled trials across Microsoft, Accenture, and a Fortune 100 firm (combined **4,867 developers**) reported an estimated **26.08% increase in completed tasks** for those using the AI tool (with additional increases in commits and compilations reported as secondary outcomes). ¹¹

However, DORA's research-driven analysis of generative AI adoption highlights a “systems” warning: a **25% increase in AI adoption** is associated with improvements in upstream engineering signals (documentation quality, perceived code quality, faster reviews/approvals), yet correlates with **lower delivery throughput (-1.5%) and lower delivery stability (-7.2%)**. ³ Interpreting this rigorously: AI can accelerate *production of change*, but without strengthened quality gates and small-batch discipline, the organization may ship more risk per unit time (or spend more time correcting/mitigating), degrading end-to-end delivery outcomes. ¹²

Quality and security risks

Security research provides concrete evidence that AI-generated code can include common vulnerabilities. A widely cited empirical security evaluation of Copilot-generated code found **~40%** of generated programs were vulnerable across tested scenarios and weaknesses. ¹³ A later targeted replication study found improvements in some contexts but still reported substantial vulnerable suggestion rates (evidence of reduction in one setting, but not elimination). ¹⁴

These findings map directly to “dark factory” risks: when AI is producing large volumes of code, the organization must treat the pipeline as a **high-throughput vulnerability generator unless controlled**—making security automation, hardened build provenance, and governance non-optional. ¹⁵

Trust, adoption, and the human factors bottleneck

DORA reports that **89% of organizations** prioritize integrating AI into applications and **76% of technologists** rely on AI for parts of their daily work. ¹⁶ Yet trust remains a barrier: DORA cites that **39% of developers** outside Google trust AI output quality only “a little” or “not at all.” ¹⁷

This trust issue is not cosmetic—it affects realized productivity. DORA describes evidence that developers who trust AI more (proxied by higher suggestion acceptance) are more productive in certain log-based analyses. ¹⁷ Complementarily, a research study on neural code completion reports that perceived productivity is driven by the **acceptance rate** of suggestions more than downstream persistence metrics—highlighting that “felt productivity” can be impacted by UI/interaction design and confidence, not only by code correctness. ¹⁸

Scrum's relevance here is that **trust-building is an organizational change problem**, not a tool-install problem—requiring transparency, shared definitions of done, and feedback loops that reduce fear and uncertainty. ¹⁹

Evidence on AI in Agile meetings and coordination

Peer-reviewed/academic work is emerging on AI assistance in Agile coordination itself. An action research study in an industrial context investigated customized LLM “meeting assistants” for **Daily Scrum** and a **feature refinement/planning meeting**, emphasizing readiness drivers, iterative prompt development, and the interaction effects on collaboration. ²⁰ This supports a key practical point: as AI automation rises, **ceremonies don't disappear**—they become more *instrumented* and more sensitive to organizational context and ethics/security constraints. ²¹

Impacts on roles and organizational design

Developer role evolution

In traditional Scrum, Developers are accountable for creating a usable increment each sprint, building the Sprint Backlog plan, and adapting daily toward the Sprint Goal while maintaining the Definition of Done. ²²

In AI-augmented settings (Levels 1–3), Developers increasingly become **designers of prompts/intent, reviewers of AI output, and builders of verification harnesses**. The “hard part” shifts toward: specifying constraints, selecting tools, reviewing diffs more strategically, and maintaining architectural coherence across faster change. This is consistent with “agentic factory” narratives where humans invest in “harness engineering” and “intent thinking.” ²³

In dark factory settings (Levels 4–5), the developer function splits into at least two specialized tracks:

“Factory engineers” (platform/harness): build the agent scaffolding, scenario libraries, evaluation gates, and secure delivery pathways (a strong match to DORA's emphasis on **quality internal platforms, strong version control**, and **working in small batches** as AI amplifiers). ²⁴

“Intent engineers” (product/requirements + technical): convert product goals into precise, testable specifications and scenarios that the agents can reliably satisfy. ⁴

Product Owner and product management

The Product Owner remains accountable for maximizing value and for Product Backlog management (ordering, transparency, communicating Product Goal). ²⁵ AI does not eliminate this; it raises the premium on it. As execution accelerates, poor prioritization produces waste faster. DORA's “AI is an amplifier” framing—AI magnifies strengths and dysfunctions—implies that weak product strategy, unclear goals, and unstable priorities will be amplified into faster thrash. ²⁶

Practical adaptation: the PO role shifts from writing many small stories to governing a portfolio of **intent packages**: problem statements, acceptance scenarios, policy constraints, and outcome metrics. This is aligned with DORA's guidance that AI success depends heavily on the surrounding system (platform, data ecosystem, user-centric focus). ²⁴

Scrum Master and agile coaching

Scrum Masters are accountable for enabling Scrum effectiveness and ensuring events occur and are productive. ²⁷ In an AI-heavy delivery environment, many “administrative” facilitation tasks can be automated (summaries, action items, risk extraction), but the **hardest parts of the role** remain stubbornly human:

conflict resolution, coaching self-management, fostering psychological safety, and aligning stakeholders amid rapid change—exactly the cultural factors DORA highlights as prerequisites for AI to create value rather than instability. ²⁸

Thus, Scrum Masters are less likely to be made obsolete than to be pressured into a higher-leverage “system steward” role: facilitator of socio-technical change, governance adoption, and continuous improvement.

QA and testing

Traditional Scrum expects quality via the Definition of Done and ongoing verification as part of the team’s work. ²⁹ With AI generating more code faster—and security studies showing non-trivial vulnerability rates—QA evolves from “test execution” to **test-and-evaluation system design**: property-based tests, scenario suites, fuzzing, regression harnesses, security scanning, and production monitoring.

In dark factories, QA becomes the design of **holdout scenarios** and automated satisfaction scoring, consistent with “scenario” and “satisfaction” language in factory narratives. ³⁰

DevOps and platform engineering

DevOps maturity becomes even more critical: AI-accelerated output increases change volume and risk. DORA’s AI Capabilities Model explicitly elevates **quality internal platforms**, **strong version control**, and **working in small batches** as capabilities that amplify positive AI impact. ²⁴ In dark factory terms, DevOps/platform becomes the “factory floor” and must be engineered for safety, provenance, and rapid rollback.

Organizationally, this tends to produce a stronger “platform team as product” model (internal developer platform), while stream-aligned teams focus on intent and outcomes.

Process changes and the fate of Scrum elements

Scrum defines a Sprint cadence (≤ 1 month) containing Sprint Planning, Daily Scrum, Sprint Review, and Sprint Retrospective. ³¹ The question is not “do these events vanish,” but “what do they become when AI changes cycle time, work packaging, and evidence collection?”

Traditional Scrum workflow vs AI-era workflow

```
flowchart TD
    subgraph TS["Traditional Scrum (human execution dominant)"]
```

```

    PBI["Product Backlog Items\n(stories, tasks)"] --> SP["Sprint
Planning\n(select + plan)"]
    SP --> DEV["Build increment\n(code+tests+review)"]
    DEV --> CI["CI/CD + verification"]
    CI --> SR["Sprint Review\n(stakeholder inspection)"]
    SR --> RETRO["Retrospective\n(process improvement)"]
    RETRO --> PBI
end

subgraph AS["AI-augmented / agentic delivery (execution accelerated)"]
    INTENT["Intent package\n(problem + constraints + scenarios)"] --> RUN["Agent
run(s)\n(generate code/tests/docs)"]
    RUN --> GATE["Automated
gates\n(unit+integration+security+policy+provenance)"]
    GATE --> DEPLOY["Deploy + observe"]
    DEPLOY --> OUTCOME["Outcome review\n(user impact + incidents + drift)"]
    OUTCOME --> LEARN["Improve harness\n(prompts, scenarios, platform)"]
    LEARN --> INTENT
end

```

The Scrum mapping is straightforward: sprint artifacts/events remain useful **if** they become the governance surface for this “intent → verify → observe → learn” loop, rather than a calendar ritual for humans writing code.

Table comparing Scrum practices to AI-era alternatives

Scrum element	Purpose	Inputs	Outputs	Cadence	Responsible roles	AI-era adaptation	Primary risks
Product Backlog	Single ordered work source ³²	Stakeholder needs; Product Goal ¹	Ordered backlog	Continuous	Product Owner ¹	Backlog items become “intent packages”: scenarios, constraints, policy tags, measurable outcomes	Amplified thrash if vague; overproduction of low-value features

Scrum element	Purpose	Inputs	Outputs	Cadence	Responsible roles	AI-era adaptation	Primary risks
Sprint	Predictable learning cycle; risk control ³³	Backlog selection	Increment + learning	1–4 weeks typical (≤ 1 month) ³³	Scrum Team ³⁴	For Level 3+: sprints may compress (“bolts”) or shift to continuous flow with explicit stage gates ³⁵	AI can introduce instability; batch size grows ³⁶
Sprint Planning	Decide Sprint Goal + plan ¹	Refined backlog; capacity	Sprint Goal + Sprint Backlog	Per sprint	Whole Scrum Team ¹	Planning becomes “intent definition + scenario design + risk classification”; capacity becomes “verification bandwidth”	False confidence; underestimated hidden verification costs; performance gaps ³⁷
Daily Scrum	Inspect progress and adapt daily plan ¹	Sprint Backlog status	Updated plan	Daily	Developers ³⁴	Replace verbal status with dashboards + exception-based sync; AI assistants can summarize risks/actions ²⁰	Overreliance on AI summaries; missing signals; human concern ³⁸
Definition of Done	Quality gate for “usable increment” ³²	Engineering standards	Acceptance criteria met	Always-on	Developers (accountable) ³⁴	Expand DoD to include security scans, provenance attestations, prompt/model version traceability	Vulnerability code generation scale ³⁹

Scrum element	Purpose	Inputs	Outputs	Cadence	Responsible roles	AI-era adaptation	Primary risks
Sprint Review	Inspect increment + adapt backlog ¹	Increment; stakeholder feedback	Updated backlog direction	Per sprint	Scrum Team + stakeholders ¹	“Outcome review”: value delivered, incident/rollback data, A/B results, model/tool changes	Shipping but wrong; misaligned success metrics
Retrospective	Improve quality & effectiveness ³²	Process data; team experience	Improvement actions	Per sprint	Scrum Team ³²	Add “factory retros”: prompt/harness quality, evaluation drift, security findings, intermittent failures	Turning into tool chasing; ignoring culture/safety

Which Scrum elements remain valuable, need adaptation, or may be obsolete

Scrum elements that remain **high-value** across AI maturity levels:

Transparency/inspection/adaptation remain foundational because AI increases speed and uncertainty; organizations need faster feedback loops, not fewer. ⁴²

Definition of Done becomes more important, not less, because AI output volume increases the probability of defects/vulnerabilities entering the system if checks are weak. ⁴³

Sprint Review (or equivalent outcome review) remains crucial; as execution becomes cheap, selecting *what to build* becomes the bottleneck. ⁴⁴

Scrum elements that strongly need **adaptation**:

Sprint Planning must incorporate AI pipeline constraints (verification capacity, policy, provenance, security) rather than only human capacity. ⁴⁵

Daily Scrum should reduce status reporting and emphasize exceptions, impediments, and risk—potentially supported by AI meeting assistants, but designed carefully to avoid overreliance. ²¹

Backlog refinement becomes closer to “continuous intent refinement,” with heavy emphasis on scenario completeness and non-functional constraints.

Scrum practices that are most “at risk” of becoming **obsolete or minimized** (often not mandated by Scrum itself):

Heavy reliance on **story-point estimation** as a planning anchor: AI changes the relationship between effort and output, and DORA data suggests that organizational performance hinges on system capabilities (platforms, batching, governance) more than “estimating better.” ⁴⁶

Meeting-heavy coordination used primarily for reporting rather than decision-making: AI can summarize state, but the organization must still preserve human-quality signals (morale, conflict, stakeholder misalignment). ⁴⁷

Governance, risk, tooling, and metrics

Governance and risk frameworks that become mandatory at higher automation

Dark-factory-like pipelines increase the importance of **AI risk governance** and **software supply chain controls**.

A practical governance stack includes:

NIST AI RMF 1.0 to structure AI risk governance (GOVERN, MAP, MEASURE, MANAGE) and trustworthiness characteristics (valid/reliable; safe; secure/resilient; accountable/transparent; explainable; privacy-enhanced; fair). ⁴⁸

OWASP Top 10 for LLM Applications to manage threats like prompt injection, insecure output handling, supply chain vulnerabilities, excessive agency, and overreliance—especially relevant when agents can act and commit code. ⁴⁹

NIST SSDF (SP 800-218) as a secure development baseline, plus NIST’s SSDF community profile work for generative AI/foundation model contexts (indicating growing recognition that AI changes secure development practices). ⁵⁰

SLSA v1.0 provenance levels to harden builds and ensure traceability of artifacts—critical when “who wrote the code” becomes “which model/toolchain generated it under what inputs.” ⁵¹

Where applicable, **EU AI Act** obligations for AI systems and general-purpose AI models strengthen the need for documentation, transparency, and risk management. ⁵²

For organizational governance maturity, **ISO/IEC 42001** provides an AI management system framework (policies, objectives, processes) for responsible development/provision/use of AI systems. ⁵³

Scrum implication: the Scrum artifact “Definition of Done” and Product Backlog metadata should explicitly encode these governance requirements (e.g., “no merge without provenance attestation,” “security scan clean,” “prompt logs retained,” “model version pinned”). ⁵⁴

Key risks intensified by dark-factory autonomy

Security vulnerabilities in AI-generated code are empirically documented, including high vulnerability rates in some studies and incomplete mitigation in replication work. ⁵⁵

“Excessive agency” and “overreliance” risks are explicitly highlighted by OWASP for LLM applications; these become acute when agents can execute actions (merge, deploy, mutate tests). ⁴⁹

Auditability and accountability become harder when code is model-generated; NIST AI RMF emphasizes transparency/accountability practices and provenance of training data and decisions as risk mitigations. ⁵⁶

Tooling shifts

Traditional Scrum tooling often emphasizes backlog management and sprint metrics. AI-era delivery requires a layered toolchain:

Developer-facing: copilot/IDE agents, code review summarizers, test generators (useful at Levels 1–2). ⁵⁷

Agent runtime + orchestration: repo navigation, tool invocation, patch generation and validation (research/OSS examples include SWE-agent; repair/issue-resolution approaches like AutoCodeRover). ⁵⁸

Verification harness: CI, integration test rigs, scenario libraries, security scanning pipelines, policy-as-code, provenance generation (aligned with SSDF/SLSA). ¹⁵

Observability: telemetry for deployments and incidents plus AI-specific logs (prompt/trace storage, model/version metadata), enabling audits and retrospectives that include “factory behavior.” ⁵⁹

Metrics and KPIs: from velocity to outcomes + flow + trust

Scrum teams often track velocity, burndown, and sprint predictability. In AI-accelerated delivery, these often become less diagnostic because output is easier to generate and the bottleneck shifts.

A better metrics stack combines:

DORA metrics (deployment frequency, lead time for changes, time to restore, change failure rate) as delivery performance standards. ⁶⁰

SPACE framework to avoid reductive productivity measurement and include satisfaction/well-being, communication/collaboration, and flow/efficiency dimensions. ⁶¹

AI-specific metrics: suggestion acceptance rate (linked to perceived productivity in code-completion research), trust surveys, “AI-generated change size,” “revert/rollback rate of AI-origin PRs,” vulnerability density for AI-generated diffs, and harness coverage (scenario pass rates, mutation testing). ⁶²

Scrum implication: Sprint Reviews should incorporate these metrics as evidence for inspection/adaptation; Retrospectives should generate improvement backlog items that target system bottlenecks (platform, batching, harness gaps), not only team habits. ⁶³

Transition strategies and concrete recommendations

Transition strategy principles grounded in recent evidence

Recent DORA guidance emphasizes that successful AI adoption is *not just tool rollout*; it requires transparency, policies, and time to learn.

DORA recommends four practical strategies to scale AI adoption: transparency about plans, addressing developer concerns, giving time to learn, and creating governing policies. ⁶⁴ It also reports very large adoption effects associated with providing developers dedicated learning time during work hours (reported as substantial relative adoption increases in their analysis). ⁶⁵ This aligns with the “trust deficit” finding that many developers do not fully trust AI output quality, which can blunt productivity gains. ¹⁷

Recommended evolution of Scrum roles

Product Owner → **Product Intent Owner**

Keep PO accountability for maximizing value, but explicitly own “intent quality”: measurable outcomes, constraints, scenario completeness, and policy tags. This is consistent with Scrum’s Product Goal focus and with “user-centric focus” as a capability that amplifies AI’s benefits. ⁶⁶

Scrum Master → **Socio-technical Flow & Governance Coach**

Maintain facilitation and continuous improvement, but prioritize: (1) psychological safety during AI change, (2) governance/policy adoption, and (3) helping teams work in small batches to counter AI-driven instability risk. ⁶⁷

Developers → **Developers + Agent Supervisors / Harness Engineers**

Split responsibilities explicitly: some focus on product-domain implementation and architecture; others on harness/platform reliability, evaluation, and security automation. This reflects the reality that AI increases change velocity and makes version control and platforms the “safety net.” ⁶⁸

QA → **Verification & Evaluation Engineering**

Shift QA goals from “test execution” to “quality system design,” using security research findings as justification for stronger automated scanning and review practices. ⁶⁹

Recommended evolution of ceremonies

Sprint Planning → **Intent Planning + Risk Classification**

Replace “how many story points” with: what intent packages can pass the current harness; what risks require additional gates; what constraints must be encoded into Definition of Done. ⁷⁰

Daily Scrum → **Exception-based coordination**

Use dashboards and AI assistants to summarize progress and risks, but keep the event focused on plan adaptation and impediment removal. Validate summaries to avoid overreliance (an OWASP-recognized LLM risk). ⁷¹

Sprint Review → **Outcome Review + Operational Evidence**

Maintain stakeholder inspection, but require evidence: DORA delivery metrics, incident/rollback data, and user impact. This counters the DORA warning that AI adoption can hurt throughput/stability even when code quality and review speed appear improved. ⁷²

Retrospective → **Dual retrospective: humans + factory**

Retros should include: harness coverage gaps, vulnerability findings, prompt/agent failure modes, and policy compliance issues, while preserving human well-being and collaboration signals (SPACE). ⁷³

Recommended metrics replacement set

Replace velocity-first thinking with a balanced scorecard:

Flow + reliability: DORA metrics as primary delivery KPIs. ⁶⁰

Human sustainability: SPACE dimensions (especially satisfaction/well-being and flow). ⁶¹

AI safety/quality: vulnerability rate for AI-assisted diffs, revert rate, change size distribution, harness pass rates, and trust measures (given documented trust gaps). ⁷⁴

Pilot operating model

A concrete, low-regret pilot is “Scrum with an AI stage-gate layer”:

Select one stream-aligned team and keep the Sprint cadence initially (2–3 weeks) to preserve learning cycles. ³¹

Redefine backlog items into intent packages: problem statement, constraints, acceptance scenarios, and required governance tags (security/provenance). ⁷⁵

Introduce a hardened “Definition of Done++”: tests + security scan + provenance + rollback plan. ⁷⁵

Measure before/after using DORA + SPACE + AI safety measures; explicitly watch for the DORA-identified pattern where upstream quality signals improve but delivery performance degrades. ⁷⁶

Scale only when DORA AI capabilities prerequisites are visible: clear AI stance, small batches, strong version control, quality internal platform, and AI-accessible internal data. ⁷⁷

Sources

Scrum Guide 2020 (Schwaber & Sutherland): Scrum pillars, roles, events, artifacts, and commitments. ⁷⁸

Agile Manifesto (official): foundational Agile values. ⁷⁹

Microsoft Research: controlled Copilot experiment reporting 55.8% faster completion. ¹⁰

Cui et al. (Feb 2025): field experiments across Microsoft/Accenture/Fortune 100 reporting ~26% task completion increases with AI tool usage. ¹¹

DORA “Impact of Generative AI in Software Development” (v2025.2): adoption rates, trust issues, upstream quality vs delivery performance impacts, and adoption strategies. ⁸⁰

DORA AI Capabilities Model (2025): “AI is an amplifier” framing and seven foundational capabilities (policy, platforms, small batches, etc.). ⁸¹

SPACE framework (Forsgren et al., ACM Queue): multidimensional developer productivity measurement. ⁶¹

SWE-bench (ICLR 2024): benchmark for evaluating models on real-world GitHub issues. ⁸²

SWE-agent and AutoCodeRover: research/OSS systems for autonomous issue resolution and program improvement. ⁵⁸

Pearce et al. (2021/2022): empirical cybersecurity evaluation of Copilot code contributions (~40% vulnerable in tested scenarios). ¹³

Majdinasab et al. (2023): targeted replication study showing improvements but persistent vulnerabilities. ¹⁴

OWASP Top 10 for LLM Applications (v1.1): key LLM security risks including prompt injection, insecure output handling, excessive agency, overreliance. ⁴⁹

NIST AI RMF 1.0 (2023): AI risk governance functions and trustworthiness characteristics. ⁸³

NIST SSDF project: SSDF baseline and AI-related community profile direction. ⁸⁴

SLSA v1.0 levels (OpenSSF): provenance levels for build integrity. ⁵¹

EU AI Act (Regulation (EU) 2024/1689) official text (EUR-Lex): EU risk-based AI regulatory framework. ⁵²

ISO/IEC 42001 standard overview: AI management system standard for responsible AI. ⁵³

Exploring Human-AI Collaboration in Agile meetings (arXiv 2024): action research on LLM assistants in Daily

Scrum and refinement/planning. ²⁰

BCG Platinion (Mar 2026): “Dark Software Factory” concept, “bolts” framing, and claimed frontier practices (industry perspective; treat as non-peer-reviewed). ³⁵

StrongDM AI (Feb 2026): “Software Factory” narrative for non-interactive agentic development (industry case study). ⁸⁵

1 6 9 19 22 25 27 29 31 32 33 34 42 43 44 54 63 66 73 75 78 scrumguides.org

<https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>

2 ¹⁰ The Impact of AI on Developer Productivity: Evidence from GitHub Copilot - Microsoft Research

<https://www.microsoft.com/en-us/research/publication/the-impact-of-ai-on-developer-productivity-evidence-from-github-copilot/>

3 16 17 36 37 41 45 64 65 67 70 72 76 80 [services.google.com](https://services.google.com/fh/files/misc/dora-impact-of-generative-ai-in-software-development.pdf)

<https://services.google.com/fh/files/misc/dora-impact-of-generative-ai-in-software-development.pdf>

4 23 ³⁵ The Dark Software Factory | Insights | BCG Platinion

<https://www.bcgplatinion.com/insights/the-dark-software-factory>

5 ⁸² SWE-bench: Can Language Models Resolve Real-World GitHub Issues?

https://arxiv.org/abs/2310.06770?utm_source=chatgpt.com

7 12 24 26 28 40 46 68 77 81 [services.google.com](https://services.google.com/fh/files/misc/2025_dora_ai_capabilities_model.pdf)

https://services.google.com/fh/files/misc/2025_dora_ai_capabilities_model.pdf

8 The Five Levels: from Spicy Autocomplete to the Dark Factory – Dan Shapiro's Blog

<https://www.danshapiro.com/blog/2026/01/the-five-levels-from-spicy-autocomplete-to-the-software-factory/>

11 [economics.mit.edu](https://economics.mit.edu/sites/default/files/inline-files/draft_copilot_experiments.pdf)

https://economics.mit.edu/sites/default/files/inline-files/draft_copilot_experiments.pdf

13 39 55 69 74 [arxiv.org](https://arxiv.org/pdf/2108.09293v1)

<https://arxiv.org/pdf/2108.09293v1>

14 [arxiv.org](https://arxiv.org/pdf/2311.11177)

<https://arxiv.org/pdf/2311.11177>

15 ⁸⁴ Secure Software Development Framework | CSRC

<https://csrc.nist.gov/Projects/ssdf>

18 ⁶² [2205.06537] Productivity Assessment of Neural Code Completion

<https://arxiv.org/abs/2205.06537>

20 21 47 71 Exploring Human-AI Collaboration in Agile: Customised LLM Meeting Assistants

<https://arxiv.org/html/2404.14871>

30 ⁸⁵ StrongDM Software Factory

<https://factory.strongdm.ai/>

38 49 OWASP Top 10 for Large Language Model Applications | OWASP Foundation

<https://owasp.org/www-project-top-10-for-large-language-model-applications/>

48 56 59 83 [nvlpubs.nist.gov](https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf)

<https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf>

50 SP 800-218, Secure Software Development Framework (SSDF) Version 1.1 ...

https://csrc.nist.gov/pubs/sp/800/218/final?utm_source=chatgpt.com

51 Security levels - SLSA

https://slsa.dev/spec/v1.0/levels?utm_source=chatgpt.com

52 Regulation - EU - 2024/1689 - EN - EUR-Lex

https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng?utm_source=chatgpt.com

53 ISO/IEC 42001:2023 - AI management systems

https://www.iso.org/standard/42001?utm_source=chatgpt.com

57 Research: quantifying GitHub Copilot's impact on developer productivity and happiness - The GitHub Blog

<https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-on-developer-productivity-and-happiness/>

58 [2405.15793] SWE-agent: Agent-Computer Interfaces Enable Automated ...

https://arxiv.org/abs/2405.15793?utm_source=chatgpt.com

60 Announcing the 2024 DORA report | Google Cloud Blog

https://cloud.google.com/blog/products/devops-sre/announcing-the-2024-dora-report?utm_source=chatgpt.com

61 The SPACE of Developer Productivity - ACM Queue

<https://queue.acm.org/detail.cfm?id=3454124>

79 Manifesto for Agile Software Development

https://agilemanifesto.org/?utm_source=chatgpt.com